

VEO Ad Pipeline

New Features — Engineer Test Plan

This release adds a guided **Simple mode** (document → draft video → high-quality video), in-app API-key management, and capability-aware degradation, while preserving the existing wizard behind an **Administrator** toggle. Branch: **feature/matt-ui**. Use the checklist below to verify each feature; mark Pass/Fail in the right-hand column.

Getting started

From the repository root, start the server (no install needed for the free paths):

```
python ui/server.py
```

Then open **http://127.0.0.1:8765**. Run the automated suite with:

```
python -m unittest discover -s ui/tests
```

Free vs paid — read before testing

Actions tagged **FREE** (building jobs, document parsing, dry-run previews, the full engine plan) cost nothing. Actions tagged **PAID** call real APIs and spend money; every one requires typing the word **SPEND** in a confirmation dialog and passes a full server-side gate sequence. You can verify the entire flow up to the SPEND dialog without spending anything — cancel there.

What the paid stages need (Administrator → Settings)

The app runs with no API keys. To exercise a real generation you need: a **Gemini** key, **ffmpeg** on PATH, and a one-time `pip install requests python-dotenv google-genai`. An **ElevenLabs** key is only for voice (presenter) jobs and is never required by Simple mode. The header chips show what is currently missing.

1 Simple mode (the new default page)

#	Feature	How to test	Expected result	Pass / Fail
1.1	Document paste	Paste scene text using Scene 1: / Scene 2: markers; click Build the job FREE .	Review section appears: correct scene count, prompts, estimated length; "Job is valid". Nothing spent.	
1.2	Document upload	Click Upload document ; choose a .txt, .md, .docx or .pdf file; Build.	File name shows with a remove button; the textarea disables; Build extracts the text and produces scenes.	
1.3	Five formats	Repeat 1.2 once per format: .txt, .md, .json, .docx, .pdf.	Each extracts readable text. Image-only PDFs report a clear "paste text instead" message (not a crash).	
1.4	AI extraction	With a Gemini key set, build from a loosely-written brief (no explicit Scene markers).	"built with: AI extraction (Gemini)"; scenes are coherent prompts derived from the brief.	
1.5	No-AI fallback	With no Gemini key (or AI extraction off in Settings), build the same brief.	"built with: basic parser" plus a warning; the app still produces a usable job.	

#	Feature	How to test	Expected result	Pass / Fail
1.6	Paste JSON artifact	Paste a raw scenes JSON array/object; Build.	"built with: JSON artifact (used as-is)"; scene count matches the JSON.	
1.7	Seed image (auto)	Build any job without supplying a seed image.	Warning notes a generated placeholder seeds scene 1; "View seed image" shows a gradient.	
1.8	Seed image (real)	Set a seed image path (Browse) to an in-repo .png/.jpg; Build.	No placeholder warning; the chosen image is used as the seed.	
1.9	Spend cap	In Review, change "Scenes to generate" before generating.	The draft confirmation reflects the chosen number of paid scenes.	
1.10	Recent jobs	Build a job, then click its chip under "Recent".	Job reopens with a freshly re-validated status (delete an artifact on disk → it shows invalid).	

2 Draft & high-quality generation PAID

#	Feature	How to test	Expected result	Pass / Fail
2.1	Draft preview	After a valid build, click Generate draft preview PAID ; type SPEND; confirm.	Per-stage progress (video → music → assemble); the draft video plays in-app on success.	
2.2	SPEND gate	Click Generate draft preview but type the wrong word, or Cancel.	Confirm button stays disabled until exactly "SPEND"; cancelling spends nothing and shows no error.	
2.3	Validation-first	Build a job, delete its storyboard.json on disk, then try to generate.	"Validation failed — nothing was spent"; no API call is made.	
2.4	HQ locked until draft	Before any draft succeeds, look at Generate high quality .	Button is disabled with a "Generate a draft first" tooltip.	
2.5	HQ pass	After a successful draft, click Generate high quality PAID ; confirm.	Runs the same prompts with the HQ model/resolution; both draft and final videos remain visible.	
2.6	Truthful status / failures	Trigger or observe a failed run (e.g. missing ffmpeg).	Failure report and stderr are shown verbatim; a music safe-fallback prompt shows a review warning.	

3 Administrator console

#	Feature	How to test	Expected result	Pass / Fail
3.1	Mode toggle	Click Administrator in the header; then ← Simple mode .	Switches between Simple and the full wizard; header chips and admin buttons change accordingly.	

#	Feature	How to test	Expected result	Pass / Fail
3. 2	Wizard intact	In Admin, walk Job → Voice → Preview → Launch.	All prior behaviour works: conditional fields, voice-pick gating, dry-run, spend gates, run history.	

4 Settings & API keys (Administrator → Settings)

#	Feature	How to test	Expected result	Pass / Fail
4. 1	Add a key	Open Settings; paste a value into Gemini or ElevenLabs; click Save .	Chip flips to “configured”. The value is never displayed back anywhere.	
4. 2	Write-only secrecy	After saving, inspect the page / network responses for the value.	The key value appears in no response, no chip, and no log — only presence booleans.	
4. 3	Clear a key	Click Clear next to a configured key.	Chip flips to “not set”; the entry is removed from the server’s .env.	
4. 4	Configurable models	Change the draft / HQ model or resolution; Save settings.	Subsequent generations use the new model/resolution (visible in the SPEND dialog).	
4. 5	Dependency readout	Read the “Engine python packages” row.	Shows which of requests / python-dotenv / google-genai are installed.	

5 Runs-without-keys & capability awareness

#	Feature	How to test	Expected result	Pass / Fail
5. 1	Runs with zero keys	With no keys configured, build a job and run a dry-run.	Building and dry-runs work; the app never hard-fails for lack of keys.	
5. 2	Capability chips	Read the header chips in Simple mode.	Chips state what each paid stage needs (e.g. “Video: add Gemini API key...”) rather than bare ticks.	
5. 3	Voice is optional	With no ElevenLabs key, use Simple mode end to end.	Voice chip says “not needed here”; nothing about Simple mode is blocked.	

6 Visual / UX

#	Feature	How to test	Expected result	Pass / Fail
6. 1	Subtler motion	Move the mouse across the background in both modes.	Fluid effect is gentle and dim (not bright/intense); pauses when the tab is hidden.	
6. 2	Larger main panel	Compare Simple mode text size to the Administrator console.	Simple mode uses noticeably larger type; the admin console stays compact.	
6. 3	Reduced motion	Enable the OS “reduce motion” setting and reload.	The fluid background is disabled; the UI is fully usable.	

7 Robustness (hardening fixes to confirm)

#	Feature	How to test	Expected result	Pass / Fail
7.1	Malformed PDF	Upload an image-only or odd PDF; Build.	Clear “could not extract / paste text instead” message — no crash, no hang.	
7.2	Oversized document	Upload a document over 20 MB.	Rejected cleanly with a size-cap message.	
7.3	Wrong file type	Try to upload a .exe or image as the document.	Rejected: “Unsupported file type...”.	
7.4	Automated suite	Run <code>python -m unittest discover -s ui/tests</code> .	All tests pass (67 at time of writing), covering parsing, gates, and key handling.	

Notes: Simple mode produces silent brand videos (music is the soundtrack — no voiceover). The assembled output always carries the generated music; the audio-free intermediate is `video/final_silent.mp4` in the run directory. All operator-supplied paths are constrained to the repository root. Runtime data (jobs, uploads, settings, run ledger, audit log) lives under `ui/data/` and is gitignored.